30-Day Android Challenge

Senior Android interview prep — Junior → Mid → Senior → Architect

2026 Edition

DAY 24

CI/CD & DevOps — Part 2: Signing, Release Automation, Rollouts & Rollback



Esmaeeil Moradi
@esmaeeil-moradi700

DAY 24

Why releasing matters in 2026

Day 23 took a commit to a green, tested build. Day 24 is the last mile: a **signed release on users' devices** — safely, repeatedly, and with a way out when something goes wrong.

- Signing keys are your app's identity — leak or lose one and updates can stop
- Releases should be boring: **build once, promote the same artifact**, automate the upload
- A shipped binary can't be recalled: **staged rollouts, halts and flags** are the fast brakes
- **In-app updates** and minimum-version gates move users off bad versions
- **Trunk-based development** and **release trains** make the cadence predictable

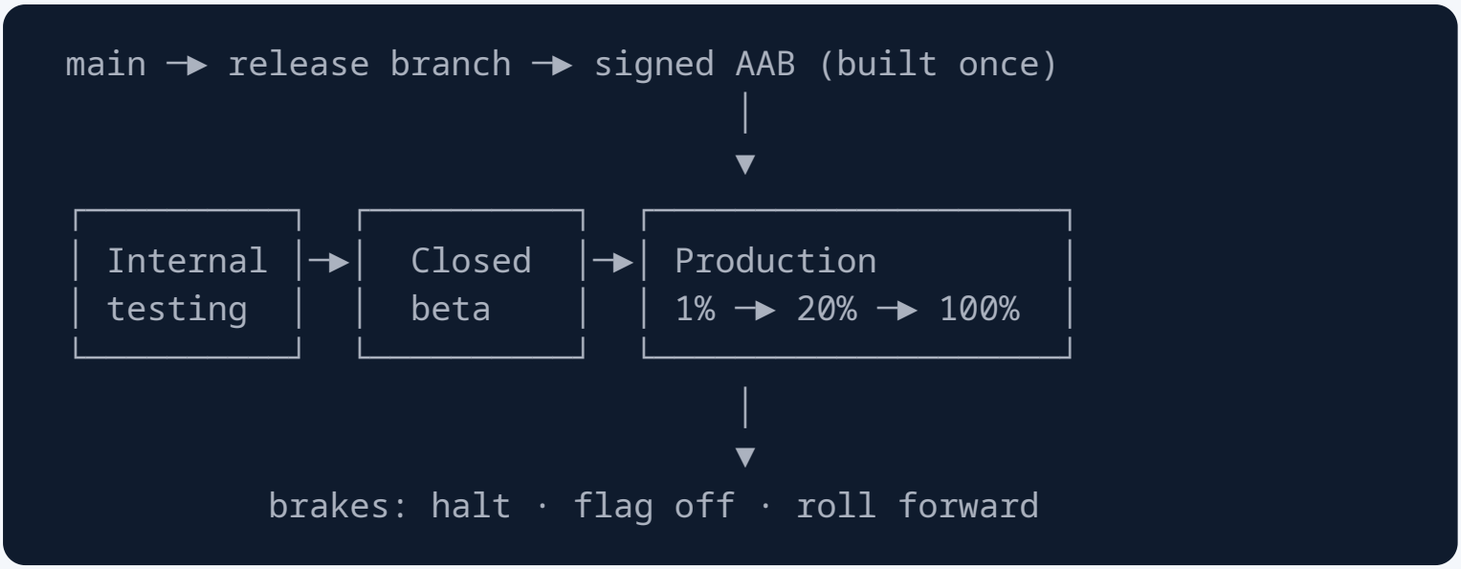
Level	Questions
Junior	Upload key vs app signing key · versionCode · Play testing tracks
Mid	Signing in CI · Release automation · Staged rollouts · In-app updates
Senior	Remote config & kill switches · "Roll it back!" · Secrets without long-lived keys
Architect	Trunk-based development · Release trains · A high-availability release pipeline



Esmaeeil Moradi
@esmaeeil-moradi700

DAY 24

Mental model: build once, promote, ramp



- **One artifact:** the AAB that passed internal testing is the one that reaches production
- **Ramp by data:** each step waits for crash, ANR and business metrics
- **No undo button:** you stop the damage and fix *forward*

THE ONE IDEA BEHIND EVERY RELEASE

Deploying a binary and releasing a feature are two different events. Keep them separate, and every release gets a brake.



Esmaeeil Moradi
@esmaeeil-moradi700

DAY 24

Junior

Upload key vs app signing key · versionCode & versionName · Play testing tracks



Esmaeeil Moradi
@esmaeeil-moradi700

Q1. Play App Signing: two keys

Key	Held by	Signs	If lost or leaked
App signing key	Google, in Play App Signing	The APKs Play generates from your AAB and delivers to users	Stays with Google; a key upgrade exists but is a rare, planned event
Upload key	You and your CI	The AAB you upload; Google verifies it, then re-signs	The account owner requests an **upload key reset** in Play Console

- Required for new apps since the App Bundle became mandatory for them (August 2021): Play builds device-specific APKs, so Play must sign them
- Debug builds use a third key — the auto-generated debug keystore



Esmaeeil Moradi
@esmaeeil-moradi700

Q1. What interviewers check

INTERVIEW TRAP

"Google Sign-In and Maps work in my release APK but fail when installed from Play." — Users get APKs signed with the **app signing key**, so its SHA-1/SHA-256 (Play Console → App integrity) must be registered in Firebase, Google Cloud and your `assetlinks.json`. Registering only the upload or debug fingerprint is the classic bug.

INTERVIEW TRAP

"Lose the keystore and the app is gone forever." — True for the old self-signed model. With Play App Signing, a lost upload key is recoverable by a reset. Keep an offline backup anyway: the reset is not instant.

FOLLOW-UP

"We also ship to other stores." — Android only installs an update signed with the **same** key. Either give Play your own app signing key at enrollment and use it everywhere, or accept that users can't move between stores without reinstalling.



Esmaeeil Moradi
@esmaeeil-moradi700

Q2. versionCode vs versionName

- **`versionCode`** — a positive integer. Every upload to an app needs a new, higher one; Play's maximum is 2100000000
- **`versionName`** — the string users see (`5.2.1`); no rules
- Android won't install a lower versionCode over a higher one — this is why mobile has **no rollback** (Q9)

```
// app/build.gradle.kts
android {
    defaultConfig {
        versionCode = providers
            .environmentVariable("VERSION_CODE")
            .map(String::toInt)
            .getOrElse(1)          // local builds
        versionName = "5.2.1"
    }
}
```



Esmaeeil Moradi
@esmaeeil-moradi700

Q2. Where the number comes from

INTERVIEW TRAP

"versionCode = `git rev-list --count HEAD`*."* — `actions/checkout` fetches only one commit by default, so the count is **1** on CI. It needs `fetch-depth: 0` — and still breaks after history rewrites.

INTERVIEW TRAP

"Use `GITHUB_RUN_NUMBER`*."* — Fine until a new workflow file or a CI migration starts again at 1. Add a fixed offset, or ask Play for the highest code in use (Gradle Play Publisher and fastlane can).

FOLLOW-UP

"Why not encode semver, like 5.2.1 → 50201?" — It leaves no room for a second build of the same version: RC2, a rebuilt hotfix, a re-upload after a failed review. Let CI own versionCode; let humans own versionName.



Esmaeeil Moradi
@esmaeeil-moradi700

Q3. Play testing tracks

Track	Audience	Use it for
Internal testing	Up to 100 testers you invite	Smoke test every release candidate; available quickly
Closed testing	Invited lists or Google Groups; several tracks possible	Beta with real users
Open testing	Anyone who joins from the store listing	Public beta at scale
Production	All eligible users, optionally staged	The real release

- **Internal app sharing:** share a build by link, no versionCode bump — also how you test in-app updates (Q7)
- New personal developer accounts must complete a closed test before production access — check the current rules

INTERVIEW TRAP

"Rebuild from the tag for production." — Then production runs a binary nobody tested. Build the AAB **once** and **promote** that exact artifact from track to track.



Esmaeeil Moradi
@esmaeeil-moradi700

DAY 24

Mid-Level

Signing in CI · Release automation · Staged rollouts & halting · In-app updates



Esmaeeil Moradi
@esmaeeil-moradi700

Q4. Signing in CI: the workflow

```yaml
bundle:
  runs-on: ubuntu-latest
  environment: release       # secrets live here
  steps:
    # checkout, JDK, setup-gradle …
    - name: Restore upload key
      run: |
        echo "$KEYSTORE_B64" | base64 -d \
          > "$RUNNER_TEMP/upload.jks"
      env:
        KEYSTORE_B64: ${{ secrets.UPLOAD_KEYSTORE_B64 }}
    - run: ./gradlew bundleRelease
      env:
        SIGNING_STORE_FILE: ${{ runner.temp }}/upload.jks
        SIGNING_STORE_PASSWORD: ${{ secrets.STORE_PASSWORD }}
        SIGNING_KEY_ALIAS: upload
        SIGNING_KEY_PASSWORD: ${{ secrets.KEY_PASSWORD }}
```

- The keystore is a binary file → store it base64-encoded as a secret, decode into the runner's temp directory
- Secrets in an **environment** can require approval and are limited to chosen branches



Esmaeeil Moradi
@esmaeeil-moradi700

Q4. Signing in Gradle

```kotlin
// app/build.gradle.kts
fun env(name: String) =
    providers.environmentVariable(name).orNull

android {
    signingConfigs {
        create("release") {
            storeFile = env("SIGNING_STORE_FILE")
                ?.let { file(it) }
            storePassword = env("SIGNING_STORE_PASSWORD")
            keyAlias = env("SIGNING_KEY_ALIAS")
            keyPassword = env("SIGNING_KEY_PASSWORD")
        }
    }
    buildTypes {
        release {
            signingConfig = signingConfigs.getByName("release")
        }
    }
}
```

INTERVIEW TRAP

Keystore in git and passwords in `gradle.properties` "because the repo is private". Every clone, laptop and fork now holds your upload key. Keys belong in CI secrets or a vault — plus one offline backup.



Esmaeeil Moradi
@esmaeeil-moradi700

Q5. Release automation: two tools, one API

	Gradle Play Publisher	fastlane `supply`
Lives in	A Gradle plugin — same build, per-variant tasks	Ruby toolchain; lanes in a `Fastfile`
Strong at	No extra runtime; listing and release notes versioned in the repo	iOS + Android in one tool; screenshots; plugins
Costs	Must keep up with AGP releases	Ruby and Bundler on every runner

- Both call the **Google Play Developer API**: open an *edit*, upload the AAB, set the track release, commit
- Pick one per repository — two tools pushing to the same app confuse everyone

INTERVIEW TRAP

"Every merge to `main` goes straight to 100% production." — That's continuous deployment without a brake. Automate the upload to internal testing; promote to production by a human or a metrics-gated job.



Esmaeeil Moradi
@esmaeeil-moradi700

Q5. In code

```
// app/build.gradle.kts
// plugins { id("com.github.triplet.play") }
play {
    track.set("internal")
    defaultToAppBundles.set(true)
}
// CI: ANDROID_PUBLISHER_CREDENTIALS holds the credentials
// ./gradlew publishReleaseBundle
```

```
# fastlane/Fastfile — promote the tested build, don't rebuild
lane :promote do
  upload_to_play_store(
    track: "internal",
    track_promote_to: "production",
    rollout: "0.05",
    skip_upload_aab: true,
    skip_upload_metadata: true
  )
end
```



Esmaeeil Moradi
@esmaeeil-moradi700

Q6. Staged rollouts and halting

- A **staged rollout** offers the release to a random percentage of eligible production users; raise it step by step (e.g., 1% → 5% → 20% → 50% → 100%)
- **Halt** stops new users from getting it. Users who already updated **keep it**
- After a halt: resume, or ship a fix with a **higher versionCode** that replaces it
- In the Play Developer API a release is `inProgress` with a `userFraction`, then `halted` or `completed`

INTERVIEW TRAP

"Halting rolls users back." — No. It only stops the spread. The users who already have the bug need a kill switch (Q8) or a fixed version (Q9).

INTERVIEW TRAP

"1% for an hour, then 100%." — Some bugs need days: migrations on old data, background work, rare devices. Gate each step on **time and volume**, and compare with the previous version at the same age.



Esmaeeil Moradi
@esmaeeil-moradi700

Q6. Automate the brake first

```
scheduled job (e.g., every 30 min)
        │
        ▼
vitals of the new versionCode
(Crashlytics, Play Developer Reporting API)
        │
        ▼
worse than the previous version?
  ├── no  → wait, or raise userFraction
  └── yes → set status "halted" via the
            Play Developer API, page on-call
```

- Halting is cheap and safe to automate; raising the percentage should need the same data **and** a human at first
- Track crash **and** ANR rates — Play's bad-behavior thresholds (Day 28) affect store visibility

> **MY TAKE**
>
> Automate the brake before you automate the accelerator.



Esmaeeil Moradi
@esmaeeil-moradi700

Q7. In-app updates: flexible vs immediate

	Flexible	Immediate
UX	Background download; the user keeps working	Full-screen; Play installs and restarts
For	Most updates	Critical fixes
You handle	Wait for `DOWNLOADED`, then call `completeUpdate()`	Resume an interrupted flow in `onResume`

- Library: `com.google.android.play:app-update` (plus `-ktx`); install progress arrives through an `InstallStateUpdatedListener`
- Signals: `updatePriority()` (0–5, set per release **only** through the Play Developer API) and `clientVersionStalenessDays()`

INTERVIEW TRAP

Still depending on the old monolithic `com.google.android.play:core`. It was split into feature libraries (`app-update`, `review`, `feature-delivery`…), and the old one is not compatible with modern targetSdk levels — migrate.



Esmaeeil Moradi
@esmaeeil-moradi700

Q7. A critical-update check

```
private val updateLauncher = registerForActivityResult(
    ActivityResultContracts.StartIntentSenderForResult(),
) { result ->
    // RESULT_CANCELED = user declined; min-version gate decides
}

private fun checkForCriticalUpdate() {
    val manager = AppUpdateManagerFactory.create(this)
    manager.appUpdateInfo.addOnSuccessListener { info ->
        val critical = info.updateAvailability() ==
            UpdateAvailability.UPDATE_AVAILABLE &&
            info.updatePriority() >= 4 &&
            info.isUpdateTypeAllowed(AppUpdateType.IMMEDIATE)
        if (!critical) return@addOnSuccessListener
        manager.startUpdateFlowForResult(
            info, updateLauncher,
            AppUpdateOptions
                .defaultOptions(AppUpdateType.IMMEDIATE),
        )
    }
}
```

INTERVIEW TRAP

"Immediate update forces the user." — The user can cancel, and it only works for installs from Google Play. Real enforcement is your own **minimum-version gate** (server or remote config) with a store link as the fallback.



Esmaeeil Moradi
@esmaeeil-moradi700

DAY 24

Senior

Remote config & kill switches · "Roll it back!" · Secrets without long-lived keys



Esmaeeil Moradi
@esmaeeil-moradi700

Q8. Remote config and kill switches

```
val config = Firebase.remoteConfig
config.setConfigSettingsAsync(
    remoteConfigSettings {
        minimumFetchIntervalInSeconds =
            if (BuildConfig.DEBUG) 0L else 3600L
    },
)
// Safe values when nothing was ever fetched (offline start)
config.setDefaultsAsync(R.xml.remote_config_defaults)
config.fetchAndActivate()

// Real-time: react to a kill switch without waiting
config.addOnConfigUpdateListener(object : ConfigUpdateListener {
    override fun onUpdate(update: ConfigUpdate) {
        if ("payments_v2_enabled" in update.updatedKeys) {
            config.activate()
        }
    }
    override fun onError(e: FirebaseRemoteConfigException) {}
})
```

- The default minimum fetch interval is 12 hours — lower it for debug builds, not blindly in production
- Typed flags, owners and experiments: Day 29. Today: flags as a **release brake**



Esmaeeil Moradi
@esmaeeil-moradi700

Q8. Designing a kill switch

- A kill switch only exists for code that shipped **with** the check — add one to every risky path (payments, new sync engine, third-party SDK) before release
- The **default** is what an offline first launch sees: new features default off; switches for existing features default on
- Fetch now, **activate at a safe point** (next launch, next screen); use the real-time listener only for emergency switches
- Anything the client receives is readable — no secrets in remote config

INTERVIEW TRAP

"We'll add a kill switch when something breaks." — The binary on millions of devices has no switch. You now need a hotfix, review and weeks of adoption.

FOLLOW-UP

"Firebase or your own service?" — Either works. What matters: cached last-known values, safe defaults, targeting by app version, an audit log of who flipped what — and a server-side fallback that doesn't depend on the client at all.



Esmaeeil Moradi
@esmaeeil-moradi700

Q9. "5.2 crashes for 3% of users. Roll it back."

Lever	Speed	Reaches	Limit
Flag / kill switch off	Minutes	Devices that fetch config	Only code behind a flag
Server-side mitigation	Minutes	Every app version	Only server-driven paths
Halt the staged rollout	Minutes	Stops new updates	Updated users keep the bug
Roll forward: hotfix with a higher versionCode	Hours to days	Everyone who updates	Review time and adoption
Push the fix: update priority, minimum version	Once the fix is live	Active users	Flexible updates can be postponed

Real incidents use several levers at once: halt + flag off now, hotfix next.



Esmaeeil Moradi
@esmaeeil-moradi700

Q9. What interviewers check

INTERVIEW TRAP

Last year's deck: *"If a metric crosses a threshold, the previous stable version must be redeployed to the affected users."* — **Impossible on Android.** Devices don't downgrade and version codes can't be reused. "Rollback" means rebuilding the **old code** with a **new, higher versionCode** — it is still a roll-forward.

INTERVIEW TRAP

Last year: *"A hotfix pipeline skips all non-essential steps."* — Skipping tests under pressure is how incident number two starts. Keep the gates; make the normal pipeline fast enough to always run them. A hotfix is a small diff, not a shortcut.

KEEP WHAT YOU NEED TO DEBUG A RELEASE

Archive the AAB, `mapping.txt` and native symbols for **every** versionCode. A crash from a build you can't deobfuscate is unreadable.



Esmaeeil Moradi
@esmaeeil-moradi700

Q10. Secrets without long-lived keys

```yaml
publish:
  needs: bundle
  runs-on: ubuntu-latest
  environment: production   # required reviewers
  permissions:
    contents: read
    id-token: write         # job may request an OIDC token
  steps:
    - uses: actions/checkout@v4
    - uses: google-github-actions/auth@v2
      with:
        workload_identity_provider: ${{ vars.WIF_PROVIDER }}
        service_account: ${{ vars.PLAY_PUBLISHER_SA }}
    # publish step runs with short-lived credentials
```

- GitHub issues an **OIDC token** naming the repo and environment (`sub`: `repo:org/app:environment:production`)
- Google Cloud **Workload Identity Federation** trusts only that subject and returns a short-lived token for the service account — no JSON key to leak or rotate



Esmaeeil Moradi
@esmaeeil-moradi700

Q10. Where each secret lives

Secret	Where	Note
Upload keystore + passwords	Environment secrets or a vault	Offline backup; never in git
Play publishing access	Workload Identity if your tool supports it; else a JSON key in environment secrets	Invited in Play Console for **one app**, only the permissions it needs
Crash-reporting upload token	CI secret	Build time only
API keys inside the app	**Not a secret** — they ship in the APK (Day 22)	Restrict by package name and signing fingerprint

INTERVIEW TRAP

"GitHub masks secrets, so they can't leak." — Masking is best-effort for exact values: a base64-decoded or transformed value prints fine. And any code in the job — including a compromised plugin — can read the environment. Keep release jobs minimal and pinned (Day 23).



Esmaeeil Moradi
@esmaeeil-moradi700

DAY 24

Architect

Trunk-based development · Release trains · A high-availability release pipeline



Esmaeeil Moradi
@esmaeeil-moradi700

Q11. Trunk-based development

	Long-lived branches (GitFlow)	Trunk-based
Integration	At the end — big merges	Small PRs into `main`, at least daily
Unfinished work	Hidden in branches	Merged, **hidden behind flags**
Release	`develop` → `release` → `main` merges	Cut a short-lived release branch from `main`
Risk	Merge hell, late surprises	Needs fast CI and flag discipline

- DORA's research lists trunk-based development among the capabilities of high-performing delivery teams

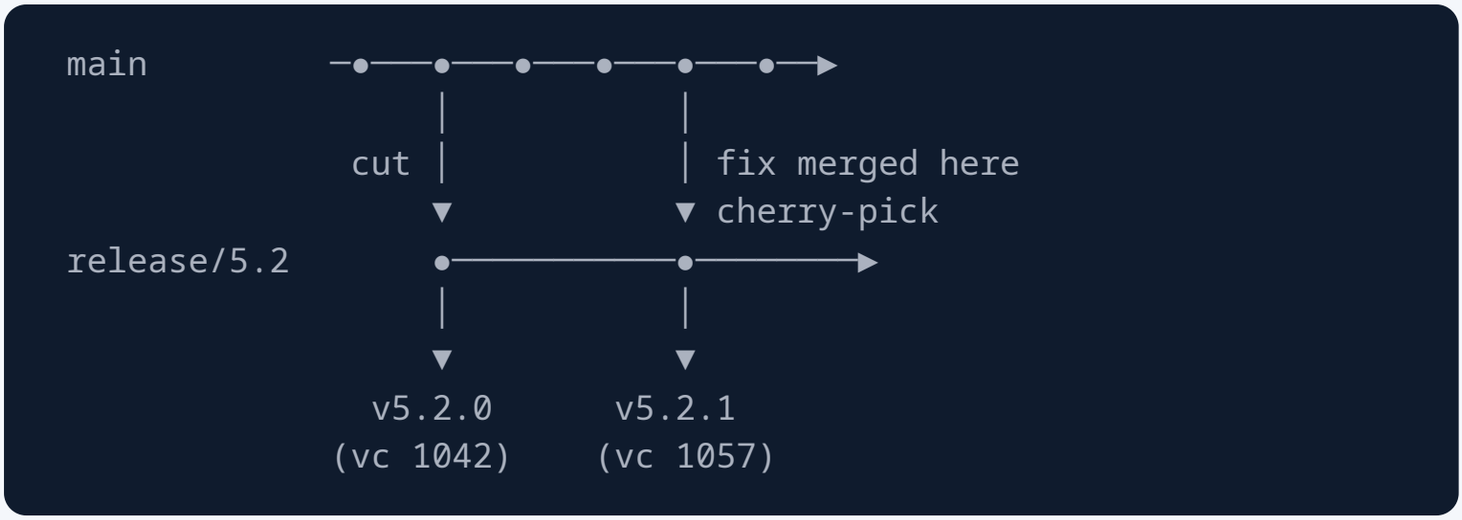
INTERVIEW TRAP

"Trunk-based means pushing to `main` without review." — No. It means **short-lived** branches: hours to a day, reviewed PRs, the CI gate from Day 23. Branch lifetime is the point, not skipping review.



Esmaeeil Moradi
@esmaeeil-moradi700

Q11. Release branches: fix on main first



- A release branch takes **fixes only** — no features
- Every fix lands on `main` first, then is cherry-picked to the release branch
- Tag each shipped build: tag ↔ versionCode ↔ mapping file

INTERVIEW TRAP

"Fix it on the release branch, merge back later." — "Later" gets forgotten, and the bug returns in the next train. Main first, cherry-pick down.



Esmaeeil Moradi
@esmaeeil-moradi700

Q12. Release trains

```
Week   1          2          3          4
5.2    on main    cut, beta  rollout    100%
5.3               on main    cut, beta  rollout
5.4                          on main    cut, beta
```

- The train leaves on a **fixed schedule** (weekly or every two weeks); the branch is cut on a fixed day
- A feature that isn't ready **misses the train** — it stays dark behind its flag and ships on the next one
- At any time: one version rolling out, one in beta, `main` moving
- A rotating **release captain** runs the checklist and makes the go/no-go call on data



Esmaeeil Moradi
@esmaeeil-moradi700

Q12. What interviewers check

INTERVIEW TRAP

"Hold the train for my feature." — One delay makes every date negotiable, and trains get bigger and riskier. Ship it dark and turn it on later.

INTERVIEW TRAP

"Weekly trains mean users are on the latest version." — Adoption lags for weeks. You'll support many versions at once: an additive API contract, server-side gating by app version, and a minimum supported version (Day 29).

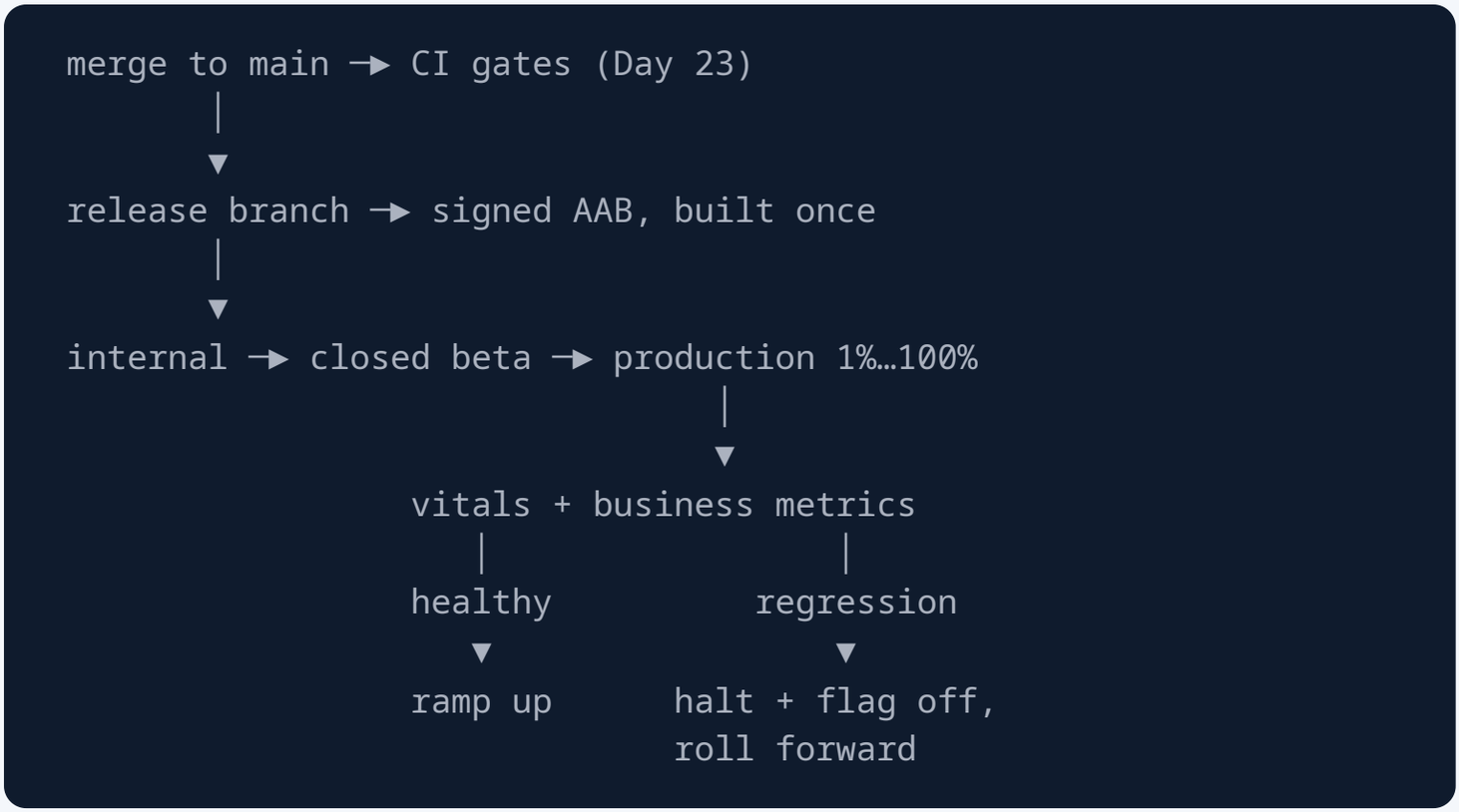
FOLLOW-UP

"How often should the train leave?" — As often as review time, QA capacity and your monitoring allow. A faster train means smaller releases, and a missed train costs less.



Esmaeeil Moradi
@esmaeeil-moradi700

Q13. Design releases for a banking app



INTERVIEW TRAP

Last year: *"blue-green deployment for mobile apps."* — You can't switch traffic between two installed binaries. The mobile equivalents: staged rollouts for the binary, flags for behavior, a versioned API on the server.



Esmaeeil Moradi
@esmaeeil-moradi700

Q13. The principles behind it

Principle	How
Key custody	App signing key with Google; upload key only in the release environment; publishing by short-lived credentials
Traceability	Tag ↔ versionCode ↔ AAB ↔ mapping file, archived per release
Human approval	Production promotion needs a reviewer; the automated brake needs none
Every risky path has a switch	Flags with safe defaults and owners; server works with every supported version
One way to ship	Hotfixes use the same pipeline and the same gates
Version floor	Minimum supported version + in-app updates move users off unsafe builds

SENIOR ANSWER IN ONE LINE

"You can't un-ship a binary: build once, promote the same artifact, ramp on data, keep a brake on every risky path — and fix forward through the same pipeline."



Esmaeeil Moradi
@esmaeeil-moradi700

DAY 24 · ARCHITECT

Day 24 — cheat sheet

- Play App Signing: Google holds the app signing key; you hold a resettable **upload key**.
- Google APIs and App Links need the **app signing key** fingerprint.
- versionCode only goes up and is owned by CI; versionName is for humans.
- Build the AAB once; promote the same artifact from track to track.
- CI signing: keystore in environment secrets, Gradle reads env vars — never in git.
- Gradle Play Publisher or fastlane: both drive the Play Developer API.
- Halting a rollout stops the spread; it doesn't roll back. Automate the brake first.
- In-app updates via `app-update`, plus your own minimum-version gate.
- Kill switches ship **before** the incident. No rollback: halt, flag off, roll forward.
- Trunk-based + release trains: fix on `main`, cherry-pick down.

Day 25 is next → Modularization & dynamic features. Save this post and follow along.



Esmaeeil Moradi
@esmaeeil-moradi700